

Formalización de teoremas



Miguel Laborda Velázquez
Trabajo de fin de grado de Matemáticas
Universidad de Zaragoza

Director : Miguel Ángel Marco Buzunáriz

Enero de 2026

Abstract

The increasing complexity of mathematical proofs and the fast development of fields such as logic and computer science have led to a growing interest in using computational tools to assist mathematicians in building and verifying rigorous proofs. A machine can assist in the proving process in two ways: first, by generating a valid proof (Automated Theorem Proving); and second, by verifying a proof provided by the user (Interactive Theorem Proving).

This thesis focuses on Lean 4, an interactive proof assistant whose mathematical community continuously updates the mathlib library, a large repository containing a considerable portion of mathematics formalised in this language.

Lean4 is based on the Calculus of Inductive Constructions (CiC), a formal framework that extends dependent type theory. Lean4 represents mathematical proofs as programs through the Curry–Howard correspondence.

This thesis briefly introduces the theoretical basis of Lean4 by giving a short overview of Type Theory, Lambda Calculus, Typed Lambda Calculus, Dependent Type Theory, and the Curry–Howard Isomorphism. In addition, it provides a brief guide on how mathematics is written in this language by introducing the tactic mode and a short list of tactics.

Finally, it contains an example of theorem proving in Lean4, showing in parallel a proof in the classical style and the corresponding code in Lean4.

Índice general

Abstract	III
1. Introducción	1
2. Marco formal detrás de Lean4	3
2.1. Teora de Tipos	3
2.2. Cálculo Lambda	4
2.3. Cálculo Lambda Tipado	5
2.4. Teoría de tipos dependientes	5
2.5. Cálculo de Construcciones Inductivas	6
2.6. Correspondencia de Curry-Howard	6
3. Escritura en Lean4 y caso práctico, unicidad salvo isomorfismo de los números reales	7
3.1. Escritura en Lean4	7
3.1.1. Algunas palabras reservadas	7
3.1.2. Tácticas	8
3.1.3. Herramientas externas utilizadas	10
3.2. Caso práctico: Unicidad de los Reales salvo isomorfía	11
3.2.1. Demostración	11
4. Conclusiones	25

Capítulo 1

Introducción

Desde mediados del siglo pasado, matemáticos y lógicos como Hilbert [11], Göedel [10], Turing [22] o Church [5] relizaron grandes avances en los campos de la lógica y la computación. Estos avances han tenido un importante papel en el desarrollo de las matemáticas y otras disciplinas, no sólo como herramienta de cálculo, sino también como apoyo en el razonamiento formal. La complejidad de los resultados matemáticos y la necesidad de garantizar su validez han supuesto un creciente interés por la exploración de métodos que permitan facilitar la formalización y verificación de demostraciones de manera rigurosa mediante herramientas computacionales.

Si bien software como *SageMath* o *RStudio* resulta muy útil a la hora de realizar cálculos, visualizar datos, estudiarlos de forma analítica y estadística y representarlos gráficamente, en este trabajo se ha querido poner el foco en la demostración formal y la verificación de demostraciones mediante asistentes de demostración. En el contexto matemático, un asistente de demostración es un software utilizado para escribir y verificar demostraciones formales basándose en un marco lógico concreto. Un marco lógico es un conjunto de normas, axiomas y lenguaje que nos indican cómo podemos trabajar con afirmaciones matemáticas de forma rigurosa. El marco clásico utilizado habitualmente es la teoría de conjuntos de Zermelo-Fraenkel con el axioma de elección (ZFC).

Hay dos formas principales en las que una máquina puede ayudar a demostrar una proposición matemática:

- Aportando una prueba (Demostración automática de teoremas). Algunos ejemplos de asistentes de demostración automática son *E*[20] y *Vampire* [14].
- Mediante la verificación de la prueba aportada (Demostración interactiva de teoremas). En este campo destacan *Coq*[21], *Lean4* e *Isabelle*[18]

Este trabajo se centra en *Lean4*. *Lean* es un asistente de demostración que busca aportar herramientas interactivas en un marco con el cual el usuario puede interactuar para generar demostraciones válidas. Fue lanzado como proyecto en 2013 por Leonardo de Moura cuando éste era investigador en Microsoft. *Lean4* es la última versión.

La lógica en la cual se basa *Lean* es el Cálculo de Construcciones Inductivas, que es una teoría de tipos. En una teoría de tipos, cada objeto (o término) tiene un tipo único. Este actúa como una “etiqueta” que restringe cómo se puede usar y combinar.

Podemos ver *Lean4* como un lenguaje de programación en sí mismo en el cual podemos escribir programas y razonar sobre las funciones creadas en los mismos.

Los objetivos del trabajo son los siguientes:

- Adquirir las habilidades necesarias en Lean4 como para poder escribir resultados matemáticos en el lenguaje.
- Realizar una pequeña introducción a Teoría de Tipos y al funcionamiento de Lean4 (última versión en el momento de la realización del trabajo).
- Introducir un ejemplo práctico de escritura y demostración de matemáticas en dicho lenguaje. El ejemplo elegido y demostrado ha sido la existencia de un isomorfismo entre cuerpos totalmente ordenados dotados del axioma del supremo. Dicho de otra forma, la unicidad salvo isomorfismos de los números reales.

Capítulo 2

Marco formal detrás de Lean4

La forma estándar de demostrar una afirmación matemática es la aportación de una prueba. Para poder realizar una necesitamos de un conjunto de axiomas y reglas en sistemas fundamentales como la lógica de primer orden con el conjunto de axiomas ZFC o el Cálculo de Construcciones Inductivas.

Es en esta última teoría en la que se fundamentan asistentes de demostración como Lean4 o Coq. Concretamente, la CCI es una extensión de la teoría de tipos dependientes. A continuación se hace una introducción a la misma. No resulta imprescindible conocer las bases de esta teoría para utilizar Lean4, pero vale la pena hacer una breve introducción.

Para poder desarrollar la siguiente sección, introducimos la siguiente notación:

Un comentario en Lean4 aparece indicado mediante `--` o escrito entre `--\-` y `--\`. Los comentarios que aparezcan a partir de ahora mostrarán la respuesta de Lean4 a cada línea de código, o aclaraciones y anotaciones sobre el código. Los dos puntos `:` indican la pertenencia de un término a un tipo, $x : X$ indica pues que x es de tipo X .

2.1. Teoría de Tipos

Cada término en la teoría es de un único tipo. Podemos pensar en los tipos como etiquetas que se asignan a objetos. Definen cómo podemos utilizar y construir objetos, así como crear otros tipos a partir de tipos básicos mediante reglas de tipado.

Tenemos tipos como `Bool` (booleanos), `Nat` (naturales) o `Prop` (proposiciones). Este último es particularmente interesante pues es el que recoge las afirmaciones matemáticas. En Lean4 podemos definir constantes y asignarles un tipo (ejemplo para los naturales, similar con booleanos, etc) mediante la palabra `def` y comprobar el tipo de un objeto mediante `#check`.

```
def m : Nat := 1
def n : Nat := 0
#check m -- m : Nat
```

El comando `variable` nos sirve para declarar variables y asignarles un tipo en el entorno en el que trabajamos. Además, podemos comprobar también que los tipos tienen su propio tipo de orden superior o universal (esto se hace para evitar paradojas de tipo Russel). De esta forma `Nat : Type`, `Type : Type 1`, `Type 1 : Type 2` etc.

```
variable (k : Nat)
#check k -- k : Nat
```

```
#check Nat      -- Nat : Type
#check Type     -- Type : Type 1
#check Type 1   -- Type 1 : Type 2
```

Podemos además construir nuevos tipos a partir de otros mediante las reglas de tipado. Ejemplos son el producto $X \times Y$ y el tipo de las funciones. Dados dos tipos A y B , el tipo $A \rightarrow B$ es el de las funciones y verifica $x : A$ y $f : A \rightarrow B$ entonces $f x : B$.

```
variable (X Y : Type)
variable {x: X}; variable {y : Y}
variable {f : X → Y}
#check X → Y      -- X → Y : Type
#check f x        -- f x : Y
#check X × Y      -- X × Y : Type
#check (x,y)      -- (x,y) : X × Y
```

En teoría de tipos no se admiten funciones con dominio parcial. Las funciones que en matemática convencional tendrían un dominio restringido se modelan reflejando las restricciones en el tipo del dominio o introduciendo dichas restricciones como hipótesis cada vez que se aplica la función. Un ejemplo típico es la división, cuyas propiedades solo pueden enunciarse y demostrarse bajo la hipótesis de que el divisor es distinto de cero. Además, mientras que en la ZFC las funciones son conjuntos (el grafo de la función), en la Teoría de Tipos se hace al revés. La noción de función es una noción primitiva y se implementan los conjuntos mediante funciones en el tipo `Prop`.

```
def evenSet : Set Nat := fun n => n % 2 = 0 -- definimos el conjunto par como la
      funcion caracteristica que va de los naturales a la proposicion n%2 = 0
#check evenSet -- Comprobamos para un numero
#check evenSet 2 -- evenSet 2 : Prop
#check evenSet 3 -- evenSet 3 : Prop
```

Veamos un poco en profundidad este enfoque. Supongamos que $A : \text{Set } X$, esto implica que si $x : X$ entonces $A x : \text{Prop}$ por la propia definición de `Set` en `mathlib` (`def Set (A : Type u) := A → Prop`)

La pertenencia tradicional de un elemento $x \in A$ se traduce pues en la aportación de una prueba de que $A x$ es cierta. Es decir, puede afirmarse que 2 es par `evenSet 2` y puede aportarse una prueba de ello, por lo que $2 \in \text{evenSet}$. Sin embargo podemos afirmar también (`evenSet 3`), pero como puede aportarse una prueba de su falsedad, la proposición es falsa y por lo tanto podemos afirmar $3 \notin \text{evenSet}$. La notación $\{x \mid \dots\}$ en Lean4 se utiliza para escribir en notación conjuntista lo anteriormente nombrado. Se puede comprobar a continuación que para Lean4 ambas formas de construir conjuntos son equivalentes.

```
example : { x : Nat | x % 2 = 0 } = fun n => n % 2 = 0 := by rfl -- por definicion
      uno es igual al otro. Mas tarde se introducirán theorem y la tactica rfl
```

2.2. Cálculo Lambda

El Cálculo Lambda es un sistema formal introducido por Alonzo Church en los años 30. Se utiliza para la definición y evaluación de funciones mediante tres constructores: las variables, la abstracción y la aplicación. Constituye uno de los cimientos de la computación moderna. Un término puede ser bien una variable x , bien una aplicación (FN) de una función F a N o bien una abstracción ($\lambda x.M$).

La abstracción es la definición de la función $\lambda x.M$, se lee como función que a cada x le asigna M . La aplicación FN evalúa la función F en el argumento N . Un ejemplo de abstracción y aplicación juntas

sería el siguiente:

$$(\lambda x.x)3$$

Lo cual se traduciría en la aplicación de la función $\lambda x.x$ aplicada al argumento 3 (evaluación de la identidad en el argumento 3).

En Lean4 se vería de la siguiente manera:

```
def f := fun x => x
#eval f 3 -- 3
```

Podemos establecer una relación entre términos mediante la β reducción (evaluación). $(\lambda x.M)N \rightarrow_{\beta} M[x := N]$. Diremos que dos términos son β iguales si podemos obtener uno a partir de otro mediante β -reducción o β -expansión (operación inversa a la reducción). También diremos que dos expresiones son α iguales si sólo difieren en el nombre de las variables.

2.3. Cálculo Lambda Tipado

El cálculo lambda tipado extiende el cálculo lambda original mediante la introducción de tipos. En el cálculo lambda sin tipos, cualquier función puede aplicarse a cualquier argumento, lo que permite auto-aplicaciones (podemos alicar un término a sí mismo), lo cual lleva a la aparición de paradojas como la de Kleene-Rosser [13].

Para evitar esto, se asigna a cada término un tipo concreto. Una función ahora tiene un tipo de la forma $N \rightarrow M$. De esta forma, una función $f : N \rightarrow M$ requiere de un elemento $m : M$ y devuelve $f m : M$. Esta restricción evita las inconsistencia nombrada anteriormente.

2.4. Teoría de tipos dependientes

A la teoría de tipos convencional se le añade la posibilidad de tener tipos variando según un término. Una familia de tipos indexada por X es una función

$$\begin{aligned} P : X &\longrightarrow \text{Type } u \\ x &\longmapsto P(x) \end{aligned}$$

Es decir, es una función que a cada término $x : X$ le asigna el tipo $P(x) : \text{Type } u$. $P(x)$ es entonces un tipo dependiente. Para todo $X : \text{Type } u$ y toda familia de tipos $P : X \rightarrow \text{Type } u$, existe el tipo de funciones dependientes de X en P , denotado por

$$\prod_{x:X} P(x).$$

Esta es la forma de implementar el cuantificador $\forall$.

```
variable (x y z : Nat)
theorem forall_ : ∀ n : Nat, n = n := by intro n; rfl
#check z = z -- z = z : Prop
#check forall_ -- forall_ (n : ℕ) : n = n
#check forall_ x -- forall_ x : x = x
#check forall_ y -- forall_ y : y = y
```

Los tipos de `forall_ x` y de `forall_ y` son distintos.

Otro ejemplo de un tipo dependiente viene dado por el $\exists$ (Σ). Este es el tipo de los pares dependientes. El segundo argumento depende del primero, como se puede ver en el ejemplo.

```
def exists_ :  $\exists$  n : Nat, n = n :=
  ⟨3, rfl⟩
#check exists_ -- exists_ :  $\exists$  n : Nat, n = n
#check exists_.2 -- exists_.2 : exists_.1 = exists_.1
```

Se observa que `exists_.2 : exists_.1 = exists_.1`, es decir, el tipo de `exists_.2` depende de `exists_.1`

2.5. Cálculo de Construcciones Inductivas

Como se ha dicho antes, la CCI es una extensión de la teoría introducida hasta ahora. Además del cálculo lambda tipado y los tipos dependientes, incorpora tipos inductivos. Un tipo inductivo es un tipo cuyos términos se construyen a partir de constructores. Es decir, los constructores son funciones que dan lugar a todos los términos del tipo mediante aplicación finita.

El ejemplo por excelencia son los números naturales. Para generarlos requerimos únicamente de dos constructores, el 0 (en Lean4 el 0 es natural) y la operación sucesor (`succ`). De esta forma, `0 : Nat` y si `n : Nat` entonces `succ n : Nat`. Podemos crear una estructura como la de los naturales gracias al `inductive`, que permite la creación de tipos inductivos a partir de los constructores que introduzcamos.

```
inductive Natt
| zero : Natt
| succ : Natt → Natt
```

Otro ejemplo son los árboles binarios:

```
inductive Tree
| leaf : Tree -- arbol con una sola hoja (nodo)
| node : Tree → Tree → Tree -- combinacion de dos arboles en uno
```

2.6. Correspondencia de Curry-Howard

La correspondencia de Curry-Howard es el isomorfismo que permite sentar las bases de Lean4 en el marco teórico de la teoría de tipos. Curry primero [8] y Howard después [12] notaron la existencia de una relación entre ciertos sistemas lógicos y la estructura de tipos de ciertos sistemas computacionales. Esta correspondencia nos permite afirmar que la demostración de una proposición equivale a la construcción de un término del tipo adecuado y esto permite verificar formalmente las pruebas dentro del sistema.

En resumen, la correspondencia de Curry-Howard establece que las proposiciones lógicas se comportan como tipos, mientras que las pruebas se comportan como términos o programas de esos tipos. Es por esto que el tipo `Prop` adquiere un papel central en Lean4: toda afirmación matemática se interpreta como un término de tipo `Prop`, y toda demostración de una proposición `P` es, a su vez, un término del tipo `P`. Podemos introducir un resultado matemático mediante el comando `lema` y comprobarlo:

```
theorem one_neq_zero: 1 ≠ 0 := by trivial
#check one_neq_zero -- one_neq_zero : 1 ≠ 0
```

Capítulo 3

Escritura en Lean4 y caso práctico, unicidad salvo isomorfismo de los números reales

Antes de la demostración del teorema se realiza una introducción a la escritura en Lean4. Lean 4 es un lenguaje de programación y, como tal, posee una sintaxis y una estructura propias.

En este capítulo repasamos las herramientas básicas necesarias para construir una prueba, haciendo una breve descripción de las principales palabras clave y tácticas. Las palabras clave son palabras reservadas del lenguaje, que tienen utilidades especiales y que no pueden utilizarse como nombres de variables. Las tácticas son las herramientas utilizadas para construir demostraciones. Modifican el objetivo y las hipótesis de los programas.

3.1. Escritura en Lean4

3.1.1. Algunas palabras reservadas

`def` y `variable`

Ambas palabras clave sirven para introducir nuevos objetos. Mientras que `def` declara objetos como funciones o constantes determinadas, el `variable` introduce parámetros globales o locales (implícitos o universales), pero que no se fijan o determinan.

```
variable (R : Type) [Ring R] -- Equivalente a "sea K un Tipo con estructura de anillo"
def f : R → R → R := fun u v => 2*(u + v) -- Dado R y su estructura podemos definir
una funcion sobre el. Definimos f como dos veces la suma de dos elementos en R
```

`theorem`, `lemma`, `proposition`, `have`, `by`

`theorem`, `lemma` y `proposition` son palabras clave que declaran una afirmación matemática. Las tres se utilizan para introducir la creación de un nuevo término cuyo tipo es la proposición que se enuncia a continuación.

Una proposición está demostrada si hemos sido capaces de construir un término cuyo tipo coincide con el de la propia proposición. Es decir, al escribir `theorem`, `proposition` o `lemma`, se declara la intención de construir una prueba de la afirmación formulada.

La palabra clave `by` introduce un bloque de demostración. Indica que vamos a utilizar el modo táctico

(demostración usando tácticas).

Por su parte, la palabra clave `have` cumple una función análoga a las tres primeras pero localmente dentro de un bloque de demostración. Mientras que éstas introducen afirmaciones a nivel global, `have` permite declarar resultados auxiliares que deben ser demostrados durante la realización de una prueba. Estos resultados, una vez demostrados, pasan a formar parte de las hipótesis disponibles.

En el InfoView (lugar en el que aparecen las metas mientras construimos la demostración) se indica el estado de la prueba al final de la línea en la que se sitúa el cursor. En este caso, en la que se introduce el `have`.

<pre>theorem examplee (P Q C : Prop) (hp : P) (hpq : P → Q) (hpqc : P → Q → C) : C := by -- usamos theorem para introducir la afirmacion have q : Q := hpq hp -- construimos q : Q -- mediante have, hpq : P → Q y hp : P exact hpqc hp q -- terminamos mediante hpqc hp p (que crea un termino de tipo C e indica que es exactamente eso lo que buscamos) #check examplee -- examplee (P Q C : Prop) (hp : P) (hpq : P → Q) (hpqc : P → Q → C) : C</pre>	<pre>1 goal P Q C : Prop hp : P hpq : P → Q hpqc : P → Q → C q : Q ⊢ C</pre>
---	--

3.1.2. Tácticas

Las tácticas en Lean4 son comandos utilizados para construir nuestras demostraciones. Para utilizar las tácticas descritas a continuación hace falta incluir el módulo táctico de Mathlib mediante

```
import Mathlib.Tactic
```

En la ventana correspondiente al InfoView aparecerá escrito el estado de la meta actual al situar el cursor sobre el final de la última línea de código. En las ventanas inferiores se escribirá la táctica correspondiente y a la derecha las hipótesis que hayan cambiado y la meta modificada. En este apartado sólo se incluyen las utilizadas durante el TFG para ilustrar el funcionamiento del modo táctico. Cabe demostrar que también se pueden realizar demostraciones sin tácticas mediante la construcción directa de un término del tipo necesario, pero la forma principal de demostración pasa por el uso de tácticas.

`apply` y `exact`

`apply e` intenta hacer coincidir la meta actual con la conclusión del tipo de `e`. Si tiene éxito, entonces la táctica devuelve tantas submetas como el número de requisitos que no han sido determinados. Pongamos que queremos demostrar `q` y que tenemos una hipótesis o lema `hp : p → q`. `apply hp` convierte nuestra meta en `p`.

`exact e` cierra la meta actual si el tipo del objetivo coincide con el de `e`.

<pre>theorem tactic_example (p q : Prop) (hp : p) (hpq : p → q) : q := by</pre>	<pre>1 goal p q : Prop hp : p hpq : p → q ⊢ q</pre>
<pre>apply hpq</pre>	<pre>⊢ p</pre>
<pre>exact hp</pre>	<pre>No goals</pre>

rfl

Se utiliza para cerrar igualdades. De esta forma, si tenemos que demostrar $h = h'$ sean lo que sean h y h' , y siempre que ambos términos de igual sea igual por definición, **rfl** cerrará la meta. De otra forma fallará.

```
example (n : Nat) : n = n := by
```

```
1 goal
n : Nat
⊢ n = n
```

```
rfl
```

```
No goals
```

intro

Se utiliza para introducir una o más hipótesis o testigos, como por ejemplo para obtener elementos concretos en un $\forall$, un conjunto o para resolver implicaciones introduciendo hipótesis del tipo correspondiente.

```
theorem example_intro1 : ∀(n : Nat), n=n := by
```

```
1 goal
⊢ ∀ (n : Nat), n = n
```

```
intro k
```

```
k : Nat
⊢ k = k
```

```
theorem example_intro2 (P : Prop) : P → P := by
```

```
1 goal
P : Prop
⊢ P → P
```

```
intro hp -- hemos obtenido hp : p, usar exact hp
```

```
hp : P
⊢ P
```

constructor

Si el objetivo de la meta principal es un tipo inductivo, **constructor** lo resuelve mediante el primer constructor que coincida. De otro modo falla. Puede utilizarse para deconstruir metas como por ejemplo $P \wedge Q$ (cuyo constructor es **And.intro**), $\exists (u : N)$ (cuyo constructor es **Exists.intro**), $P \iff Q$, (que tiene como constructor el **Iff.intro**), $P \vee Q$ (ya que la disyunción posee dos constructores, el **Or.inl** y el **Or.inr**).

```
theorem example_constructor (p q : Prop) (hiff :
  p → q ∧ q → p): p ↔ q := by
```

```
1 goal
p q : Prop
hiff : p → q ∧ q → p
⊢ p ↔ q
```

```
constructor
```

```
2 goals
case mp
p q : Prop
hiff : p → q ∧ q → p
⊢ p → q
case mpr
p q : Prop
hiff : p → q ∧ q → p
⊢ q → p
```

cases', **by_cases** y **rcases**

Se utilizan para trabajar por casos, descomponiendo hipótesis o proposiciones en otras más simples.

by_contra

Se utiliza para hacer demostraciones por reducción al absurdo. Se niega la meta y se introduce como hipótesis. La nueva meta pasa a ser `False`

```
theorem example_by_contra (n m k : Nat) (hnmk : n
  < m ∧ m < k) : n < k := by
```

```
1 goal
n m k : ℕ
hnmk : n < m ∧ m < k
⊢ n < k
```

```
by_contra hc
```

```
hc : ¬n < k
⊢ False
```

obtain

La táctica `obtain` se utiliza para extraer información a partir de una hipótesis, introduciendo nuevas variables o hipótesis en el la demostración

```
theorem example_obtain (n : Nat) (hm : ∃ m, 0 < m
  ∧ m < n) : 0 < n := by
```

```
1 goal
n : ℕ
hm : ∃ m, 0 < m ∧ m < n
⊢ 0 < n
```

```
obtain⟨m, h0m, hmn⟩ := hm
```

```
n m : ℕ
h0m : 0 < m
hmn : m < n
⊢ 0 < n
```

Más tácticas

Existen también tácticas que automatizan ciertos procesos mecánicos como simplificaciones algebraicas. Algunas de ellas son: `linarith` (para desigualdades lineales) `simp`, `ring` (anillos), `field_simp`, `field`.

Por poner algún ejemplo de funcionamiento, `simp` aplica a la meta actual lemas marcados con `[@simp]` en Mathlib para intentar simplificar la expresión. Por otro lado, tácticas como `ring` identifican si están en una estructura determinada (en el caso de `ring` un anillo conmutativo) y utilizan las propiedades o simplificaciones básicas (conmutatividad, distributividad etc) para resolver la meta.

Si bien estas tácticas funcionan muchas veces, fallan o no son aplicables en algunos casos (como por ejemplo aquellos que impliquen división y una prueba intermedia de que el numerador no se anula) y son estos los que más tiempo lleva probar.

Para ver una explicación más detallada sobre el modo táctico, se recomienda visitar la sección de tácticas de la página web de Lean4 *Theorem Proving in Lean 4* [6]

3.1.3. Herramientas externas utilizadas

Mathlib [7] es una biblioteca construida por la comunidad de Lean4 en la que viene recogida toda la matemática formalizada en este lenguaje. Mediante la importación de la misma en nuestro código podemos acceder a multitud de teoremas, lemas, proposiciones y definiciones ya escritas. Mediante las mismas podemos crear y extender nuevas definiciones y clases, así como el utilizar resultados ya existentes para ayudarnos en la construcción de nuestras demostraciones, ahorrando gracias a ello mucho tiempo y esfuerzo que de no tenerla disponible invertiríamos en demostrar lemas y proposiciones previas más sencillas o no relevantes para nuestro trabajo.

El problema de Mathlib radica en la búsqueda de los resultados. Contiene una enorme cantidad de contenido creado por la comunidad y es complicado seguir las normas que dictan los nombres de los enunciados (que tienden a ser descriptivos). Es por ello que hay varias herramientas que resultan muy útiles a la hora de encontrar lo que buscamos en Mathlib. A la hora de realizar el trabajo se han utilizado 3, que son LeanSearch [9], Moogler [17] y ChatGPT. Los dos primeros son buscadores semánticos sobre Mathlib. Esto quiere decir que no sólo comparan palabras clave sino que tratan de interpretar el significado de la entrada para realizar la búsqueda. Aceptan entradas tanto en lenguaje natural como en formato LaTeX y código de Lean. El uso de ChatGPT se ha limitado a la búsqueda de resultados en la biblioteca Mathlib, ya que su desempeño a la hora de la escritura de programas en Lean4 que funcionen no es satisfactorio.

Conviene nombrar la existencia del `exact?` y del `apply?`. Ambas son tácticas que tratan de cerrar o simplificar la meta actual mediante la búsqueda en Mathlib de resultados relevantes o aplicables a la meta actual, automatizando ligeramente el proceso de búsqueda. No se ha recurrido demasiado a su uso por no resultar tan útiles cuando se requieren varios lemas o pasos no triviales para cerrar la meta.

Para el aprendizaje de los fundamentos de escritura en Lean4 se ha recurrido al *Natural Number Game* [3] (uno de los juegos interactivos de Lean4) y al curso *Formalising Mathematics 2024* impartido por Kevin Buzzard [2]. Ambos realizan una introducción al modo táctico y facilitan la comprensión del mismo al comenzar a trabajar con el mismo.

3.2. Caso práctico: Unicidad de los Reales salvo isomorfía

A partir de ahora vamos a ver cómo se implementa un ejemplo de demostración. Dado un cuerpo con orden total y el axioma del supremo (todo conjunto no vacío y acotado superiormente tiene supremo, que es la menor de las cotas superiores) lo llamaremos cuerpo real. Probaremos la isomorfía entre cualesquiera dos cuerpos reales.

El programa completo y sin interrupciones se incluye en el Anexo A para facilitar su revisión. También se incluye el código íntegro en el siguiente repositorio de Github [15]¹. En esta sección se expone la prueba realizada de forma convencional intercalada por fragmentos de código considerados relevantes (se han seleccionado de entre las 1060 líneas totales del código original). En muchos casos se sustituirán por `sorry` las demostraciones poco importantes (`sorry` permite utilizar aquellos resultados en los que se escriba sin la necesidad de aportar una prueba. No se ha utilizado en el código original).

3.2.1. Demostración

Definición (Cota superior). Sea $(X, \leq)$ un conjunto ordenado y sea $A \subseteq X$. Diremos que un elemento $M \in X$ es una cota superior de A si

$$\forall a \in A, \quad a \leq M.$$

Definición (Supremo). Sea $(X, \leq)$ un conjunto ordenado y sea $A \subseteq X$. Un elemento $s \in X$ se llama supremo de A si se cumplen las siguientes condiciones:

1. s es una cota superior de A ;
2. Para toda cota superior M de A , se tiene $s \leq M$.

En este caso escribimos $s = \sup A$.

¹<https://github.com/Miiike1/proyecto>

Axioma 1 (Axioma del supremo). Sea $(X, \leq)$ un conjunto totalmente ordenado. Diremos que X satisface el axioma del supremo si todo subconjunto $A \subseteq X$ no vacío y acotado superiormente admite supremo, es decir,

$$\forall A \subseteq X, (A \neq \emptyset \wedge \exists M \in X, \forall a \in A, a \leq M) \Rightarrow \exists s \in X, s = \sup A.$$

Definición (Cuerpo real). Diremos que un cuerpo $(X, +, \cdot)$ es un cuerpo real si está dotado de un orden total $\leq$ compatible con las operaciones y satisface el axioma del supremo.

Esta definición la podemos identificar con la creación de la clase `RealField`:

```
class RealField (R : Type) extends SupSet R, Field R, LinearOrder R,
  IsStrictOrderedRing R where
  sSup_axiom :  $\forall (S : \text{Set } R), S.\text{Nonempty} \rightarrow \text{BddAbove } S \rightarrow \text{IsLUB } S (s\text{Sup } S)$ 
```

Notar tres cosas:

1. Una clase en Lean4 es una estructura usada para describir propiedades u operaciones matemáticas asociadas a tipos. Permite que Lean infiera estas estructuras automáticamente al utilizarlas sobre un tipo si existe una instancia adecuada. Una instancia `instance` de una clase implementa las operaciones o estructuras recogidas para un tipo concreto. Por ejemplo `class Field` recoge la estructura algebraica de cuerpo, mientras que a tipos como `Nat` o `Int` se les puede dotar de la operación suma mediante la instancia `Add Nat` o `Add Int`, que recoge cómo la suma se implementa en cada caso.
2. En Mathlib el supremo se define simplemente como una aplicación de los subconjuntos de X en X , esto debido a que, como se ha comentado antes, las funciones no pueden definirse en un dominio parcial. Es el axioma `sSup_axiom` el que afirma que si $S : \text{Set } R$ y S es no vacío y acotado superiormente, entonces el supremo es la menor de las cotas superiores de R .
3. Al crear nuestra clase `RealField` a partir de `Field R`, `LinearOrder R` y `IsStrictOrderedRing R` incluimos las estructuras de cuerpo, de relación de orden lineal y de anillo con orden total. Asociados a ellas incluimos también todos los resultados demostrados en Mathlib y podremos utilizarlos sobre objetos de la nueva clase que estamos creando.

Para probar el teorema necesitamos una serie de propiedades y lemas previos. La primera de ellas es la propiedad arquimediana. Notar que asumimos que poseemos una estructura isomorfa a $\mathbb{N}$ dentro de un cuerpo totalmente ordenado (igual para $\mathbb{Q}$ y $\mathbb{Z}$). Asumiremos pues que $n \in \mathbb{N}$ puede verse como elemento de nuestro cuerpo gracias a la coerción. La coerción es una aplicación $\uparrow: \mathbb{N}, \mathbb{Z}, \mathbb{Q} \rightarrow \mathbb{K}$ (eleva la estructura de los naturales, enteros o racionales al cuerpo imagen $\mathbb{K}$ correspondiente).

Lema 1. Dado $A \subseteq X$, X cuerpo real, A no vacío y acotado superiormente, si $b \in X, b < \sup A$ entonces $\exists a \in A$ tal que $b < a$.

Demostración. Por contradicción. Si no existe ningún elemento en A menor que b , entonces b es una cota superior de A y por tanto mayor o igual que el supremo de A . $\square$

Lema 2 (Propiedad arquimediana). $\forall x, y \in X$ con X cuerpo real, $y > 0$, $\exists n \in \mathbb{N}$ tal que $x < n \cdot y$.

Demostración. Comenzamos tomando $x, y \in X$ con $0 < y$. Supongamos que no es cierto. Esto querría decir que $\nexists n \in \mathbb{N}$ tq $x < ny$, lo cual equivale a que $\forall n \in \mathbb{N}, n \cdot y \leq x$.

Supongamos ahora $x \leq 0$ tomamos $n = 1$ y vemos que la afirmación anterior no se cumple, pues $x < 0 < 1 \cdot y$. Es decir, que de ser cierta la afirmación, x sólo puede ser positivo.

Supongamos ahora $0 < x$. Definimos el conjunto $A = \{z \in X \mid \exists m \in \mathbb{N}, z = m \cdot y\}$. Tenemos que el

conjunto no está vacío, pues $y \in A$, $y = 1 \cdot y$. También que A está acotado por x ($\forall n \in \mathbb{N}, n \cdot y \leq x$). Sea $S = \sup A$. Por el axioma del supremo, es la menor de las cotas superiores de A . Tenemos además que $S - y < S$ porque $y > 0$. Por lo tanto, por el Lema1 obtenemos que $\exists \alpha \in A$, $S - y < \alpha$. Como $\alpha \in A$, $\exists n_0 \in \mathbb{N}$ tal que $\alpha = n_0 \cdot y \implies S - y < n_0 \cdot y \implies S < (n_0 + 1) \cdot y$. Como $(n_0 + 1) \cdot y \in A$, hemos encontrado un elemento de A mayor que $S = \sup A$, lo cual es una contradicción. $\square$

Parte del código correspondiente a esta demostración se expone y explica a continuación:

```
instance (X : Type) [RealField X] : Archimedean X where
  arch := by
```

En Lean4, la propiedad arquimediana se formaliza mediante una instancia de la clase `Archimedean`, que proporciona la prueba de que un tipo con clase `RealField` cumple la propiedad, de modo que Lean lo reconoce `RealField` automáticamente como arquimediano.

```
intro x y hy1
by_contra hc; push_neg at hc -- hc : ∀ (x_1 : ℕ), x_1 · y < x
by_cases hx : x ≤ 0
· specialize hc 1
  rw[one_smul] at hc; linarith
```

Tras tomar dos testigos $x \ y : X$ e introducir la hipótesis $hy1 : 0 < y$ se procede por contradicción `by_contra hc`. Negando la meta, buscamos demostrar `False`. Resolvemos utilizando el 1 en `hc` (cuyo tipo se ha indicado al final de la línea) para hallar la contradicción con `specialize hc 1`.

```
· push_neg at hx
let A := {(z : X) | ∃ m : ℕ, z = m · y}
have Anone : A.Nonempty := by use y; use 1; rw[one_smul] -- A no vacío
have Abdd : BddAbove A := by sorry -- A acotado
obtain ⟨a, ha1, ha2⟩ := x_lt_supA_a_lt_sup Anone Abdd (sSup A - y)
  (by exact sub_lt_self (sSup A) hy1) -- (a : X) (ha1 : a ∈ A) (ha2 : sSup A - y < a)
obtain ⟨n, hn⟩ := ha1 -- (n : ℕ) (a = n · y)
```

Para el caso $x > 0$ se define A , se demuestra que es no vacío y acotado y se aplica el teorema:

```
theorem x_lt_supA_a_lt_sup {A : Set X} (hA : A.Nonempty) (hB : BddAbove A)
  (b : X) : b < sSup A → ∃ a ∈ A, b < a := by sorry
```

Se termina la demostración de la propiedad obteniendo la contradicción, dada por la obtención de las siguientes dos hipótesis : `sup_lt : sSup A < (n + 1) · y` y `n_add_one_lt_sup : (n + 1) · y sSup A`. `linarith` demuestra `False` por ser ambas incompatibles.

```
have sup_lt: sSup A < (n + 1) · y
· rw[hn, sub_lt_iff_lt_add] at ha2
  rw[succ_nsmul]; exact ha2
have n_add_1_in_A: (n + 1) · y ∈ A := by use (n + 1)
have n_add_one_lt_sup:= supA_gt_a_in_A Anone Abdd
specialize n_add_one_lt_sup ((n + 1) · y) n_add_1_in_A
linarith
```

A partir de la propiedad arquimediana podemos deducir la densidad de los racionales en cualquier cuerpo real.

Lema 3 (Densidad de $\mathbb{Q}$). *Dado X cuerpo real, $\forall x, y \in X$ con $x < y$, $\exists p \in \mathbb{Q}$ tal que $x < p < y$.*

Demostración. Comenzamos demostrando la proposición para $0 < x, y$. Primero notar que para cualquier $x \in X$ con $x > 0$, $\exists n \in \mathbb{N}$ tal que $0 < \frac{1}{n} < x$ como consecuencia (casi) directa de la propiedad arquimediana. Tenemos pues que existe un $n_0 \in \mathbb{N}$ tal que $\forall n \in \mathbb{N}$ si $n_0 \leq n$ entonces $0 < \frac{1}{n} < y - x$. Sea $t_0 = \frac{1}{n_0}$. Evidentemente, $0 < t_0 < y - x$ y por lo tanto $x + t_0 < y$.

Como consecuencia de la propiedad arquimediana se puede probar que $m = \max\{n \in \mathbb{N} \mid n \cdot t_0 < x\}$ verifica $m \cdot t_0 \leq x < (m + 1) \cdot t_0$. Tomando $p = (m + 1) \cdot t_0$ tenemos el resultado requerido, puesto que $m \cdot t_0 < x < (m + 1) \cdot t_0 < x + t_0 < y$. $\square$

En Mathlib encontramos el teorema `exists_rat_btwn`, que formaliza este resultado dada una instancia de `Archimedean`. Podemos pues obtenerlo directamente de Mathlib.

```
theorem Q_is_dense (R : Type) [RealField R] (x y : R) (h : x < y) :
  ∃ p : ℚ, x < p ∧ p < y := by
  exact exists_rat_btwn h
```

A continuación se definen tres aplicaciones. La tercera de ellas será el isomorfismo entre cuerpos que buscamos.

Definición (Funciones Q y Q^0). *Dado X cuerpo real definimos Q_X como la función*

$$Q_X : X \longrightarrow \mathcal{P}(\mathbb{Q})$$

$$x \longmapsto \{q \in \mathbb{Q} \mid \uparrow q < x\}.$$

Sea $X_0 = \{x \in X \mid 0 < x\}$. Definimos Q_X^0 como:

$$Q_X^0 : X_0 \longrightarrow \mathcal{P}(\mathbb{Q})$$

$$x \longmapsto \{q \in \mathbb{Q} \mid 0 < q \wedge \uparrow q < x\}.$$

En Lean4 se les ha llamado `Qlower` y `Qlower_gt0`. En ambas dos hace falta indicar de qué cuerpo partimos (`Qlower R x` funciona, `Qlower x` no). Podría haberse dejado como variable implícita, pero para evitar fallos de inferencia de tipos se ha preferido dejarlo como argumento explícito.

```
def Qlower (R : Type) [RealField R] : R → (Set ℚ) := fun x => {(q : ℚ) | ↑ q < x}

def Qlower_gt0 (R : Type) [RealField R] : R → (Set ℚ) := fun x => {(q : ℚ) | 0 < q ∧
  q ∈ Qlower R x}
```

Definición (Función ϕ). *Dados dos cuerpos reales definimos ϕ_{XY} como la función:*

$$\phi_{XY} : X \longrightarrow Y$$

$$x \longmapsto \phi_{XY}(x) = \sup\{q \in Y \mid q \in Q_X(x)\}.$$

Es decir, dados dos cuerpos reales X e Y definimos ϕ como el supremo en Y de los racionales menores a $x \in X$. En ambas funciones anteriores, el subíndice recoge primero el cuerpo de salida y en el caso de ϕ a continuación el de llegada.

Es fácil probar $\forall x \in X, Q_X(x) \neq \emptyset$. También es fácil ver que este mismo conjunto es acotado superiormente en Y , por lo que $\phi_{XY}(x)$ está bien definido y es la menor de las cotas superiores de $Q_X(x)$ visto como subconjunto de Y .

```
def ϕ (X Y : Type) [RealField X] [RealField Y] :
  X → Y := fun x => sSup {(q : Y) | q ∈ Qlower X x}
```

Las dos funciones siguientes se han utilizado en el programa para probar la biyectividad de ϕ_{XY} . Para cada $x \in X$ corresponden al conjunto $Q_X(x) \subseteq X$ y a ϕ_{XX} , por lo que son algo redundantes.

```
def SR (X : Type) [RealField X] : X → (Set X) := fun x => {(q : X) | q ∈ Qlower X x}
def Supx (X : Type) [RealField X] : X → X := fun x => sSup (SR X x)
```

Lema 4. *Dado X cuerpo real, las funciones Q_X y Q_X^0 definidas anteriormente son inyectivas. Esto implica $x = y \iff Q_X(x) = Q_X(y)$.*

Si además $0 < x \wedge 0 < y \implies (x = y \iff Q_X(x) = Q_X(y) \iff Q_X^0(x) = Q_X^0(y))$.

Demostración. Supongamos que Q_X no es inyectiva. Es decir, existe al menos un par $x, y \in X$ tal que $x \neq y$ pero $Q_X(x) = Q_X(y)$. Suponemos que $x < y$. Por la densidad de $\mathbb{Q}$, $\exists p \in \mathbb{Q}$ tal que $x < p < y$. Tenemos pues que p está en $Q_X(y)$ pero no en $Q_X(x)$. Contradicción. Análogo para $y < x$. La demostración para Q^0 es muy similar. Ambas cadenas de implicaciones son directas una vez tenemos la inyectividad de ambas funciones. $\square$

El código de este teorema viene separado en 4 resultados distintos. Se incluye aquí parte del código de la demostración de que Q_X es inyectiva:

En este primer fragmento se introducen las hipótesis y se indica que se va a proceder por contradicción.

```
lemma Qlower_inj (R : Type) [RealField R] : Function.Injective (Qlower R) := by
  intro x y h
  rw[Qlower, Qlower] at h
  simp at h
  by_contra hc; push_neg at hc
```

La táctica `by_cases` divide en dos casos la meta. El primero considera `hc1 : x < y` y el segundo `hc1 : ¬x < y`. A partir de ahí se busca demostrar que ambos conjuntos son distintos encontrando un racional entre x e y y comprobando que está en uno y no en otro. Por la similitud entre ambas partes de la demostración, el caso `hc1 : ¬x < y` lo sustituimos por `sorry`.

```
by_cases hc1: x < y
· obtain ⟨q, hqx, hqy⟩ := Q_is_dense R x y hc1 -- (q : ℚ) (hqx : x < ↑q) (hqy : ↑q < y)
  have hqinSQY: q ∈ Qlower R y := by
    exact hqy
  have hqnoinSQX: q ∉ Qlower R x := by
    rw[Qlower]
    simp
    linarith
  have diff_sets: Qlower R x ≠ Qlower R y := by
    rw[Qlower, Qlower]
    simp
    rw[Set.ext_iff]
  -- Set.ext_iff.{u} {α : Type u} {a b : Set α} : a = b ↔ ∀ (x : α), x ∈ a ↔ x ∈ b
    push_neg
    use q
    right
    constructor
    · exact hqnoinSQX
    · exact hqinSQY
  trivial
· sorry
```

Los otros tres son los siguientes:

```

theorem x_eq_y_iff_QlowerRx_eq_QlowerRy {R : Type} [RealField R] (x y : R) :
  sorry
lemma Qlower_determines_Qlowergt0 {R : Type}
[RealField R] {x y : R} : (0 < x) → (0 < y)
  → Qlower R x = Qlower R y → Qlower_gt0 R x = Qlower_gt0 R y := by
  sorry
theorem x_eq_y_iff_Qlowergt0Rx_eq_Qlowergt0Ry
(R : Type) [RealField R] (x y : R) : (0 < x) →
(0 < y) → (x = y ↔ Qlower_gt0 R x = Qlower_gt0 R y) := by
  sorry --una vez obtenidos los tres anteriores, este se demuestra usandolos

```

Lema 5. Sean $p, q \in \mathbb{Q}$ y $x \in X$, con X e Y cuerpos real. Se cumple $p <_X x \wedge x \leq_X q \implies p <_Y q$.

Lema 6. Sean X e Y dos cuerpos reales. Se cumple: $Q_X(x) = Q_Y(\phi_{XY}(x))$.

Demostración. En esta prueba hay que tener cuidado con el cuerpo en el que operamos. Por ello, denotaremos $<_X$ si relacionamos elementos de X y $<_Y$ si relacionamos elementos de Y . Tomamos $q \in Q_X(x)$, vamos a ver que $q \in Q_Y(\phi_{XY}(x))$. Notar primero que $\phi_{XY}(x)$ es el supremo y por lo tanto es la menor de las cotas superiores del conjunto $Q_X(x)$ visto en Y (es no vacío y acotado). Tenemos que $q <_X x$ y queremos probar que $q <_Y \phi_{XY}(x)$. Esta desigualdad es sencilla, puesto que por tener $q \in Q_X \subseteq \mathbb{Q}$ tenemos directamente que $\uparrow q \in Q_X \subseteq Y$, y por ser $\phi_{XY}(x)$ el supremo de dicho conjunto en Y , obtenemos la desigualdad.

Ahora, tomando $q \in Q_Y(\phi_{XY}(x))$ queremos demostrar $q \in Q_X(x)$, es decir, sabemos que $q <_Y \phi_{XY}(x)$ y queremos probar $q <_X x$. Lo probamos por contradicción.

Supongamos que $x \leq_Y q$. Tenemos pues que $\forall p \in Q_X(x)$, $p <_X x \leq_X q$. Aplicando el Lema5 obtenemos que $\forall p \in Q_X(x)$, $p \leq_Y q$. Es decir, q es una cota superior del conjunto $Q_X(x)$ en Y , y por lo tanto, mayor que el supremo del mismo que es $\phi_{XY}(x)$. Por lo tanto $\phi_{XY}(x) < q$ y llegamos a la contradicción. De forma parecida se prueba la parte (2) $\square$

El esquema de la prueba en Lean4 de este resultado se recoge a continuación.

En el fragmento siguiente, tras introducir notación y resultados demostrados anteriormente, se toma un testigo q tal que $q \in \mathbb{Q}$.

```

theorem QlowerRx_eq_QlowerZF_phiRZx (R Z : Type) [RealField R] [RealField Z] (x : R) :
  Qlower R x = Qlower Z (phi R Z x) := by
  rw[Qlower, Qlower]
  rw[Set.ext_iff]
  let set := { y : Z | ∃ (q : ℚ), (↑q < (x : R)) ∧ (↑q : Z) = y }
  let sup := sSup { y : Z | ∃ (q : ℚ), (↑q < (x : R)) ∧ (↑q : Z) = y }
  have set_eq_set : set = { y : Z | ∃ (q : ℚ), (↑q < (x : R)) ∧ (↑q : Z) = y } := by rfl
  have sup_eq_sup : sup = sSup { y : Z | ∃ (q : ℚ), (↑q < (x : R)) ∧ (↑q : Z) = y } := by
    rfl
  have nonempt2 := rats_lt_sup_rats_nonempt R Z x -- racionales < x no vacio
  have bddabv2 := rats_lt_sup_rats_bddabv R Z x -- acotado
  have sup_IsLUB := sSup_axiom set nonempt2 bddabv2 --menor de las cotas superiores
  intro q; simp

```

El objetivo en este punto es probar $\vdash \uparrow q <_X x \leftrightarrow \uparrow q < \phi R Z x$. Utilizamos la táctica `constructor` para dividir el objetivo en dos metas: $\vdash \uparrow q <_X x \rightarrow \uparrow q < \phi R Z x$ y $\vdash \uparrow q < \phi R Z x \rightarrow \uparrow q <_X x$.

Esta es la primera parte:

```

constructor
· intro hq

```

```

rw[φ, Qlower]
have q_in_set: (↑q : Z) ∈ set
· rw[set_eq_set]; simp; exact hq
have q_lt_sup : (↑q : Z) < sup := by sorry
rw[sup_eq_sup] at q_lt_sup
exact q_lt_sup

```

A partir de aquí se demuestra la segunda parte por contradicción (`by_contra` niega la meta con la introducción de la hipótesis `hc : ¬↑q < x` y cambia la meta a `false`)

```

· intro hq; simp at hq
  rw[φ, Qlower] at hq
  by_contra hc; simp at hc
  · have q_gt_sup: sup ≤ q
      sorry
      rw[sup_eq_sup] at *
      have: sSup { y:Z | ∃ q ∈ {q:Q | ↑q < (x : R)}, (↑q : Z) = y } = sup := by rfl
      rw[this] at hq
      linarith -- resuelve false debido a que tenemos q < sup y sup ≤ q

```

Lema 7. Sean X e Y cuerpos reales. $\phi_{YX} = \phi_{XY}^{-1}$, por lo tanto ϕ_{YX} es biyectiva.

Demostración. Se obtiene directamente del lema anterior. Si $x \in X$ entonces $Q_X(x) = Q_Y(\phi_{XY}(x))$.

Componemos y vemos que obtenemos la identidad:

$$\phi_{YX} \circ \phi_{YX}(x) = \phi_{YX}(\phi_{YX}(x)) = \sup\{q \in X \mid q \in Q_Y(\phi_{YX}(x))\} = \sup\{q \in X \mid q \in Q_X(x)\} = x$$

La última igualdad es fácil de verificar. Tenemos pues que ϕ_{YX} es inversa por la izquierda de ϕ_{XY} . Razonando análogamente obtenemos que también es la inversa por la derecha. La biyectividad es consecuencia directa de la existencia de inversa. $\square$

La demostración de resultado está dividido en dos y se utilizan los siguientes lemas auxiliares. Remarcar que `Supx_is_idd2` prueba que `Supx R x = x`, con `Supx R` coincidiendo con ϕ_{XX}

```

lemma compo (R Z : Type) [RealField R] [RealField Z]
  (x : R) : (φ Z R ∘ φ R Z) x = φ Z R (φ R Z x) := rfl

lemma gt_sup_if_upbd {R : Type}
  [RealField R] (x : R) (A : Set R) (ha : ∀ a ∈ A, a ≤ x) :
  A.Nonempty → sSup A ≤ x := by sorry

lemma Supx_is_idd2 (R : Type) [RealField R] :
  ∀(x:R), Supx R x = x := by sorry

theorem φφx_eq_x (R Z : Type) [RealField R] [RealField Z] :
  ∀(x : R), ((φ Z R) ∘ (φ R Z)) x = x := by sorry

```

El primer resultado demuestra $\phi_{YX} = \phi_{XY}^{-1}$. Utilizamos para ello el resultado anterior `QlowerRx_eq_QlowerZF_φRZx` (los racionales menores que x coinciden con los racionales menores que $\phi_{XY}(x)$). El segundo prueba que ϕ_{XY} es biyectiva utilizando `Function.bijective_iff_has_inverse` (una función es biyectiva si tiene inversa) y probando que tiene inversa a izquierda y derecha.


```

theorem  $\phi$ ZR_is_ $\phi$ RZ_inverse (R Z : Type)
[RealField R] [RealField Z] : ( $\phi$  Z R)  $\circ$  ( $\phi$  R Z) = id := by
  funext x
  have equal := QlowerRx_eq_QlowerZF_ $\phi$ RZx R Z x -- equal : Qlower R x = Qlower Z ( $\phi$ 
    R Z x)
  rw[id, compo,  $\phi$ ,  $\leftarrow$ equal, Qlower]
  exact Supx_is_idd2 R x

theorem  $\phi$ _is_bijective (R Z : Type) [RealField R] [RealField Z] :
Function.Bijective ( $\phi$  R Z) := by
  rw [Function.bijective_iff_has_inverse]
  use ( $\phi$  Z R) --afirmamos que esta es la inversa
  constructor -- divide ser inversa en ser inversa a derecha e izquierda
  · intro x
    rw[ $\leftarrow$ compo]
    exact  $\phi\phi$ x_eq_x R Z x
  · intro z
    rw[ $\leftarrow$ compo]
    exact  $\phi\phi$ x_eq_x Z R z

```

Definición (Suma y producto de conjuntos en $\mathbb{Q}$). Sean $A, B \subseteq \mathbb{Q}$. Definimos la suma de conjuntos en $\mathbb{Q}$ ($\oplus$) como:

$$A \oplus B = \{q \mid q = a + b, a \in A \wedge b \in B\}.$$

De forma análoga, definimos el producto ($\otimes$):

$$A \otimes B = \{q \mid q = a \cdot b, a \in A \wedge b \in B\}.$$

Ambos aparecen en el programa como las funciones `addset` y `mulset`.

```

def addset {R : Type} [Ring R] :
(Set R)  $\rightarrow$  (Set R)  $\rightarrow$  (Set R) := fun U V => {(x : R) |  $\exists$  u  $\in$  U,  $\exists$  v  $\in$  V, x = u + v
}

def mulset {R : Type} [Ring R] :
(Set R)  $\rightarrow$  (Set R)  $\rightarrow$  (Set R) := fun U V => {(x : R) |  $\exists$  u  $\in$  U,  $\exists$  v  $\in$  V, x = u * v
}

```

Lema 8. Dado $x, y \in X$ cuerpo real, se cumple: $Q_X(x+y) = Q_X(x) \oplus Q_X(y)$.

Demostración. Probar que $Q_X(x) \oplus Q_X(y) \subseteq Q_X(x+y)$ es sencillo. Sea $q \in Q_X(x) \oplus Q_X(y)$. Tenemos que $\exists a \in Q_X(x) \wedge b \in Q_X(y)$ tal que $q = a + b$. Como $a < x \wedge b < y$ se obtiene que $q = a + b < x + y$ y por lo tanto $q \in Q_X(x+y)$.

Para probar $Q_X(x+y) \subseteq Q_X(x) \oplus Q_X(y)$ sea $q \in \mathbb{Q}$, $q < x+y$ queremos demostrar que $\exists a < x \wedge b < y$ tal que $q = a + b$.

Notar que $(x+y-q)/2 > 0$ y que por lo tanto $x - (x+y-q)/2 < x$. Por la densidad de los racionales, $\exists p \in \mathbb{Q}, x - (x+y-q)/2 < p < x$. Es fácil ver que tomando $a = p$ y $b = q - p$ obtenemos el resultado requerido.

A continuación el código que recoge esta demostración. Lo probamos utilizando el teorema `Set.ext_iff.mpr`, que afirma que dos conjuntos A y B son iguales si y sólo si $a \in A \leftrightarrow a \in B$.

```

lemma Qlower_x_addset_Qlower_y_eq_Qlower_x_add_y {R : Type} [RealField R] (x : R) (y
  : R) :
  addset (Qlower R x) (Qlower R y) = Qlower R (x + y) := by
  rw[addset, Qlower, Qlower, Qlower]
  apply Set.ext_iff.mpr

```


Para descomponer la doble implicación en dos utilizamos `constructor`. La primera meta $x1 < x$ y $x2 < y \implies \uparrow x1 + \uparrow x2 < x + y$ se prueba fácilmente con `Rat.cast_add` (la coerción preserva la suma) y la táctica `linarith`. Para la segunda queremos demostrar $\exists a < x \wedge b < y$ tal que $q = a + b$. Comenzamos obteniendo p con $x - (x + y - \uparrow k)/2 < p < x$ gracias al resultado `Q_is_dense`. Usamos p y $k - p$ mediante `use p` y `use k-p`.

```
intro k
constructor
· simp; intro x1 hx1 x2 hx2 hk
  rw[hk, Rat.cast_add]; linarith
· simp; intro h
  have ineq : (x + y - ↑k)/2 > 0 := by linarith
  have : x - (x + y - ↑k)/2 < x := by linarith
  obtain ⟨p, hp1, hp2⟩ := Q_is_dense R (x - (x + y - ↑k)/2) x this
  use p
  constructor
  · exact hp2
  · use k - p
    constructor
    · rw[Rat.cast_sub]; linarith
    · simp
```

Lema 9. Dado $x, y \in X$ cuerpo real, $0 < x, 0 < y$, se cumple: $Q_X^0(x \cdot y) = Q_X^0(x) \otimes Q_X^0(y)$.

Demostración. Probar que $Q_X(x) \otimes Q_X^0(y) \subseteq Q_X^0(x \cdot y)$ es sencillo. Sea $q \in Q_X(x) \otimes Q_X^0(y)$. Tenemos que $\exists a \in Q_X^0(x) \wedge b \in Q_X^0(y)$ tal que $0 < a$ y $0 < b$, y tal que $0 < q = a \cdot b$. Como $0 < a < x \wedge 0 < b < y$ se obtiene que $0 < q = a \cdot b < x \cdot y$. Por lo tanto $q \in Q_X^0(x \cdot y)$.

Para probar $Q_X^0(x \cdot y) \subseteq Q_X^0(x) \otimes Q_X^0(y)$ sea $q \in Q_X^0(x \cdot y)$, $0 < q < x \cdot y$. Queremos demostrar que $\exists a, b \in \mathbb{Q}$ tal que $0 < a < x \wedge 0 < b < y$ tal que $q = a \cdot b$.

Notar que $(x \cdot y)/q > 0$ (puesto que $0 < q < (x \cdot y)$) y que por lo tanto $0 < x/((x \cdot y)/q) < x$. Por la densidad de los racionales, $\exists p \in \mathbb{Q}, 0 < x/((x \cdot y)/q) < p < x$. Es fácil ver que tomando $a = p$ y $b = q/p$ obtenemos el resultado requerido.

La idea de esta demostración es muy parecida a la de la suma, pero en Leann4 es más larga porque debemos asegurar en todo momento que los elementos que tomemos estén acotados inferiormente por 0. El lema correspondiente en el programa es el siguiente:

```
lemma Qlower_gt0_preserves_mulset {R : Type} [RealField R] {x : R} {y : R} :
  (0 < x) → (0 < y) → mulset (Qlower_gt0 R x) (Qlower_gt0 R y) = Qlower_gt0 R
  (x*y) := by sorry
```

Lema 10. Sean X e Y dos cuerpos reales. Dados $x, y \in X$, $\phi_{XY}(x + y) = \phi_{XY}(x) + \phi_{XY}(y)$.

Demostración. Por el Lema4 tenemos que:

$$\phi_{XY}(x + y) = \phi_{XY}(x) + \phi_{XY}(y) \iff Q_Y(\phi_{XY}(x + y)) = Q_Y(\phi_{XY}(x) + \phi_{XY}(y)).$$

Partiendo del término de la derecha y aplicando el Lema8 y el Lema6 obtenemos:

$$Q_Y(\phi_{XY}(x) + \phi_{XY}(y)) = Q_Y(\phi_{XY}(x)) \oplus Q_Y(\phi_{XY}(y)) = Q_X(x) \oplus Q_X(y) = Q_X(x + y) = Q_Y(\phi_{XY}(x + y)).$$

Por lo tanto $Q_Y(\phi_{XY}(x) + \phi_{XY}(y)) = Q_Y(\phi_{XY}(x + y))$ y por el primer si y sólo si hemos demostrado el teorema. $\square$

El fragmento de código se escribe a continuación. Al igual que en nuestra demostración hecha a mano, se demuestra aplicando los lemas previos (mediante la táctica `rw`).

```

lemma  $\phi$ _preserves_add (R Z : Type) [RealField R] [RealField Z] (x y : R) :
   $\phi$  R Z (x + y) =  $\phi$  R Z x +  $\phi$  R Z y := by
  rw[x_eq_y_iff_QlowerRx_eq_QlowerRy,  $\leftarrow$  QlowerRx_eq_QlowerZF_ $\phi$ RZx]
  rw[ $\leftarrow$ Qlower_x_addset_Qlower_y_eq_Qlower_x_add_y]
  rw[ $\leftarrow$ Qlower_x_addset_Qlower_y_eq_Qlower_x_add_y]
  rw[ $\leftarrow$  QlowerRx_eq_QlowerZF_ $\phi$ RZx,  $\leftarrow$  QlowerRx_eq_QlowerZF_ $\phi$ RZx]

```

Lema 11. Se cumplen las dos siguientes propiedades:

1. $\phi_{XY}(0_X) = 0_Y$.
2. $\forall x \in X, \phi_{XY}(-x) = -\phi_{XY}(x)$.

Demostración. (1): $\phi_{XY}(0_X) = \phi_{XY}(0_X + 0_X) = \phi_{XY}(0_X) + \phi_{XY}(0_X) \iff \phi_{XY}(0_X) = 0_Y$.

(2): $\phi_{XY}(0) = \phi_{XY}(x + (-x)) = \phi_{XY}(x) + \phi_{XY}(-x) = 0_Y \iff \phi_{XY}(-x) = -\phi_{XY}(x)$. $\square$

Corresponden con los lemas `zero_to_zero` y `$\phi$ minusx_eq_minus $\phi$ x`.

```

lemma zero_to_zero (R Z : Type) [RealField R] [RealField Z] :  $\phi$  R Z 0 = 0 := by
  sorry
lemma  $\phi$ minusx_eq_minus $\phi$ x {R Z : Type} [RealField R]
  [RealField Z] {x : R} :  $\phi$  R Z (-x) = - ( $\phi$  R Z x) := by
  sorry

```

Lema 12. Dados $x, y \in X$, $x < y$ se cumple $\phi_{XY}(x) < \phi_{XY}(y)$.

Idea de la demostración: Directo a partir del hecho de que dados $x, y \in X$ cuerpo real, $x < y \iff Q_X(x) \subseteq Q_X(y)$.

$$x <_X y \iff Q_X(x) \subset Q_X(y) \iff Q_Y(\phi_{XY}(x)) \subset Q_Y(\phi_{XY}(y)) \iff \phi_{XY}(x) <_Y \phi_{XY}(y).$$

Corresponde al lema `$\phi$ _preserves_order`. Para demostrarse se utiliza el lema previo `Q_lower_preserves_order`.

```

lemma Qlower_preserves_order {R : Type} [RealField R] (x y : R) :
  ((Qlower R x)  $\subseteq$  (Qlower R y))  $\leftrightarrow$  (x < y) := by sorry

lemma  $\phi$ _preserves_order (R Z : Type) [RealField R]
  [RealField Z] (x y : R) : x < y  $\leftrightarrow$   $\phi$  R Z x <  $\phi$  R Z y := by
  constructor
  · intro hxy
    apply (Qlower_preserves_order x y).mpr at hxy
    apply (Qlower_preserves_order ( $\phi$  R Z x) ( $\phi$  R Z y)).mp
    rw[ $\leftarrow$ QlowerRx_eq_QlowerZF_ $\phi$ RZx R Z x,  $\leftarrow$ QlowerRx_eq_QlowerZF_ $\phi$ RZx R Z y]; exact hxy
  · sorry

```

Lema 13. $0 < x \implies Q_X^0(x) = Q_Y^0(\phi_{XY}(x))$.

Demostración. Se prueba de forma muy similar al Lema6, pero teniendo cuidado con tomar sólo racionales positivos usando además que ϕ_{XY} preserva el orden.

```

lemma supZ_Qlower_gt0Rx_eq_ $\phi$ RZx (R Z : Type) [RealField R] [RealField Z] (x : R) :
  0 < x  $\rightarrow$  sSup { y : Z |  $\exists q \in Qlower\_gt0$  R x, y = (q : Z) } =  $\phi$  R Z x := by sorry

theorem Qlower_gt0Rx_eq_Qlower_gt0Z $\phi$ RZx (R Z : Type) [RealField R] [RealField Z] (x :
  R) : (0 < x)  $\rightarrow$  Qlower_gt0 R x = Qlower_gt0 Z ( $\phi$  R Z x) := by sorry

```

Cabe destacar que en este conjunto de teoremas que trabajan la igualdad de conjuntos he tenido algún problema con la inferencia de tipos. Lean 4 tiene capacidad para deducir automáticamente los tipos en muchas ocasiones, lo cual agiliza el trabajo porque permite evitar en muchas situaciones la escritura. Esto permite, por ejemplo, que dado $q : \mathbb{Q}$, podamos escribir $q < x$ con $x : X$ sin utilizar la aplicación $\uparrow$. En el caso de este tipo de teoremas, llegué a expresiones de conjuntos que parecían iguales pero que tenían distinto tipado. Por ejemplo, Lean reconocía el conjunto $\{y \mid \exists q \in \{q \mid \uparrow q < x\}, y = \uparrow q \wedge 0 < q\}$ como subconjunto de X y yo requería que fuera subconjunto de Y .

Lema 14. *Dados $x, y \in X$, $\phi_{XY}(x \cdot y) = \phi_{XY}(x) \cdot \phi_{XY}(y)$*

Demostración. Primeramente demostramos el lema para $0 < x, 0 < y$.

Podemos proceder análogamente al caso de la suma, buscamos utilizar el Lema4:

$$\phi_{XY}(x \cdot y) = \phi_{XY}(x) \cdot \phi_{XY}(y) \iff Q_Y^0(\phi_{XY}(x \cdot y)) = Q_Y^0(\phi_{XY}(x) \cdot \phi_{XY}(y))$$

Queremos usar el Lema9. Para ello primero debemos probar $0_Y < \phi_{XY}(x), 0_Y < \phi_{XY}(y)$. Estas dos desigualdades se obtienen directamente de Lema11.1 y del Lema12. Usando ahora sí el Lema9:

$$Q_Y^0(\phi_{XY}(x) \cdot \phi_{XY}(y)) = Q_Y^0(\phi_{XY}(x)) \otimes Q_Y^0(\phi_{XY}(y))$$

Utilizando la ecuación (1) y el hecho de que $0 < \phi_{XY}(x) \wedge 0 < \phi_{XY}(y)$ (de nuevo porque ϕ_{XY} preserva el orden):

$$\begin{aligned} Q_Y^0(\phi_{XY}(x)) \otimes Q_Y^0(\phi_{XY}(y)) &= Q_X^0(x) \otimes Q_X^0(y) = Q_X^0(x \cdot y) = Q_Y^0(\phi_{XY}(x \cdot y)) \\ \implies Q_Y^0(\phi_{XY}(x) \cdot \phi_{XY}(y)) &= Q_Y^0(\phi_{XY}(x \cdot y)) \end{aligned}$$

Utilizando el Lema4 de nuevo se obtiene el resultado.

Los casos en los que $x = 0 \vee y = 0$ son triviales por el Lema11.1. Los casos en los que $x < 0 \vee y < 0$ se obtienen fácilmente a partir del resultado para $0 < x \wedge 0 < y$ y el Lema11.2. $\square$

El código correspondiente se incluye a continuación. Como se puede observar, es mucho más extenso al correspondiente a la suma. Esto se debe a las restricciones de positividad impuestas. También supuso un reto por la confusión que suponía el trabajar con él. A la hora de escribirlo, sabía que tenía todas las piezas para demostrarlo pero encajarlas me supuso un día y medio de trabajo. Además, la construcción del teorema exigió también la demostración del lema auxiliar `supZ_Qlower_gt0Rx_eq_0RZx`, el cual prueba que el supremo en el cuerpo imagen de los racionales mayores que cero y menores que x (elemento positivo del dominio) coincide con su imagen por la función ϕ . Es uno de los fragmentos de código más largos (ocupa 142 líneas de código, cerca del 13 % del total) y por eso no se comenta.

En esta primera parte se prueba que todos los términos con los que se va a trabajar son positivos:

```
lemma  $\phi$ _preserves_mul_x_y_pos (R Z : Type) [RealField R] [RealField Z] (x y : R)
  : (0 < x)  $\rightarrow$  (0 < y)  $\rightarrow$ 
   $\phi$  R Z (x * y) =  $\phi$  R Z x *  $\phi$  R Z y := by
  intro hx hy
  have mul_gt_0 : 0 < x * y := by exact mul_pos hx hy
  have aux1 := Qlower_determines_Qlowergt0 hx hy
  have cero_lt_ $\phi$ xy : 0 <  $\phi$  R Z (x * y)
  · rw[ $\leftarrow$  zero_to_zero R Z]
    exact ( $\phi$ _preserves_order R Z 0 (x * y)).mp mul_gt_0
  have cero_lt_ $\phi$ x : 0 < ( $\phi$  R Z x)
```

```

· rw[← zero_to_zero R Z]
  exact (φ_preserves_order R Z 0 (x)).mp hx
have cero_lt_φy : 0 < (φ R Z y)
· rw[← zero_to_zero R Z]
  exact (φ_preserves_order R Z 0 y).mp hy

```

A continuación se introducen nuevas hipótesis utilizando lemas previamente demostrados (principalmente `Qlower_gtORx_eq_Qlower_gtOZφRZx_1`, el correspondiente al Lema13 y `Qlower_gt0_preserves_mulset`, el correspondiente al Lema9)

```

have cero_lt_φxφy : 0 < φ R Z x * φ R Z y
· exact mul_pos cero_lt_φx cero_lt_φy
have aux2 := Qlowergt0_determines_Qlower
  (φ R Z (x*y)) (φ R Z x * φ R Z y) cero_lt_φxy cero_lt_φxφy
rw[x_eq_y_iff_QlowerRx_eq_QlowerRy]
apply aux2
have Qlower_gtORx_eq_Qlower_gtOZφRZx_1 := Qlower_gtORx_eq_Qlower_gtOZφRZx R Z (x*y)
mul_gt_0
have Qlower_gtORx_eq_Qlower_gtOZφRZx_2 := Qlower_gtORx_eq_Qlower_gtOZφRZx R Z x hx
have Qlower_gtORx_eq_Qlower_gtOZφRZx_3 := Qlower_gtORx_eq_Qlower_gtOZφRZx R Z y hy
have cero_lt_sup1 : 0 < sSup {y : Z | ∃ q ∈ Qlower_gt0 R x, y = ↑q} := by
  rw[supZ_Qlower_gtORx_eq_φRZx]; exact cero_lt_φx; exact hx
have cero_lt_sup2 : 0 < sSup {z : Z | ∃ q ∈ Qlower_gt0 R y, z = ↑q} := by
  rw[supZ_Qlower_gtORx_eq_φRZx]; exact cero_lt_φy; exact hy
have preserved1 := Qlower_gt0_preserves_mulset hx hy
have preserved2 := Qlower_gt0_preserves_mulset cero_lt_sup1 cero_lt_sup2

```

Procedemos como con el caso de la suma, utilizando mediante la táctica `rw` las hipótesis introducidas anteriormente con `have`

```

rw[←Qlower_gtORx_eq_Qlower_gtOZφRZx_1]
rw[← supZ_Qlower_gtORx_eq_φRZx R Z, ← supZ_Qlower_gtORx_eq_φRZx R Z]
· rw[← preserved1, ← preserved2]
  rw[supZ_Qlower_gtORx_eq_φRZx, supZ_Qlower_gtORx_eq_φRZx]
  · rw[Qlower_gtORx_eq_Qlower_gtOZφRZx_2, Qlower_gtORx_eq_Qlower_gtOZφRZx_3]
  · exact hy
  · exact hx
· exact hy
· exact hx

```

Las líneas anteriores solamente recogen el código para el caso $0 < x \wedge 0 < y$. A continuación se nombran los teoremas que recogen el caso $0 < x \wedge 0 < y$ y el caso $(x < 0 \wedge 0 < y) \vee (0 < x \wedge y < 0)$. Utilizan que $\forall x \in X, \phi_{XY}(-x) = -\phi_{XY}(x)$ (demostrado en Lema11.2) y el lema anterior.

```

lemma φ_preserves_mul_x_y_neg (R Z : Type) [RealField R] [RealField Z] (x y : R)
  : (x < 0) → (y < 0) → φ R Z (x * y) = φ R Z x * φ R Z y := by sorry

lemma φ_preserves_mul_diff_sign (R Z : Type)
  [RealField R] [RealField Z] (x y : R) : (x < 0 ∧ 0 < y) ∨ (0 < x ∧ y < 0) → φ R Z
  (x * y) = φ R Z x * φ R Z y := by
  intro or
  cases' or with h1 h2 ; sorry; sorry -- deconstruye la hipotesis or en el caso (x <
    0 ∧ 0 < y) y el caso (0 < x ∧ y < 0)

```

El lema que unifica los tres anteriores es `φ_preserves_mul`. Se divide en casos según el signo de x y posteriormente según el signo de y . La táctica usada para esto ha sido `by_cases`.

```

theorem  $\phi$ _preserves_mul (R Z : Type)
  [RealField R] [RealField Z] (x y : R) :  $\phi$  R Z (x * y) =  $\phi$  R Z x *  $\phi$  R Z y := by
  by_cases hxlt : x < 0
  · by_cases hylt : y < 0
    · sorry
    · sorry

```

Lema 15. $\phi_{XY}(1_X) = 1_Y$

Demostración. Se prueba de forma análoga al Lema11.1 pero jugando con el 1 en lugar de con el 0 y con el hecho de que el producto es preservado por ϕ_{XY} $\square$

El lema que recoge esta demostración es `one_to_one`:

```

theorem one_to_one (R Z : Type) [RealField R] [RealField Z] :  $\phi$  R Z 1 = 1 := by sorry

```

Teorema (Unicidad de los Reales salvo Isomorfismo). *Cualesquiera dos cuerpos reales X e Y son isomorfos.*

Demostración. Basta ver que la función ϕ_{XY} es un isomorfismo de cuerpos totalmente ordenados. Para ello basta ver que cumple todas las condiciones: Es biyectiva (Lema7. Preserva el orden (Lema12. Preserva la suma (Lema10). Preserva el producto (Lema9). $\phi_{XY}(0_X) = 0_Y$ (Lema11.1) y $\phi_{XY}(1_X) = 1_Y$ (Lema15) $\square$

Para demostrar este teorema simplemente se recurre a los resultados obtenidos previamente. La táctica `constructor` se utiliza repetidamente para separar la meta (construida mediante $\wedge$), mientras que `exact` la va cerrando recurriendo a los lemas que prueban la biyectividad de la función y la preservación de las operaciones, el orden y los elementos neutros para las operaciones.

```

theorem Real_fields_are_isomorphic (X Y : Type) [RealField X] [RealField Y] :
   $\exists$  f : X  $\rightarrow$  Y,
    Function.Bijective f  $\wedge$ 
    ( $\forall$  x y, f (x + y) = f x + f y)  $\wedge$ 
    ( $\forall$  x y, f (x * y) = f x * f y)  $\wedge$ 
    ( $\forall$  x y, x < y  $\leftrightarrow$  f x < f y)
     $\wedge$  ((f 0 = 0)  $\wedge$  (f 1 = 1)) := by

  use  $\phi$  X Y
  constructor
  · exact  $\phi$ _is_bijective X Y
  · constructor
    · exact  $\phi$ _preserves_add X Y
    · constructor
      · exact  $\phi$ _preserves_mul X Y
      · constructor
        · exact  $\phi$ _preserves_order X Y
        · constructor
          · exact zero_to_zero X Y
          · exact one_to_one X Y

```


Capítulo 4

Conclusiones

Durante este trabajo se han perseguido tres objetivos. El primero de ellos pasaba por la familiarización con el lenguaje de programación y la adquisición del nivel de manejo necesario como para poder escribir definiciones, teoremas y demostraciones. El segundo consistía en la adquisición de conocimientos básicos sobre la teoría de tipos en la que se basa Lean4, y el tercero consistía en la formalización de un teorema en Lean que tradujera la demostración realizada por el alumno.

Se considera que los objetivos planteados al inicio del trabajo se han cumplido de manera satisfactoria. Gracias a la utilización de recursos como *Natural Number Game* y el curso *Formalising Mathematics*, así como de la realización del caso práctico, se ha conseguido ganar soltura en el uso de Lean4. Los conocimientos adquiridos permiten la escritura y comprensión de definiciones, proposiciones y demostraciones formales, así como la interacción con el asistente de demostración de forma práctica.

El estudio de la teoría de tipos ha permitido comprender mejor los fundamentos lógicos del sistema, lo que ha resultado útil para abordar con éxito la formalización del teorema considerado.

Además, el uso continuado de Lean4 ha permitido observar las diferencias existentes entre la demostración matemática tradicional y la verificación formal mediante el uso de software especializado en el que nos vemos obligados a especificar pasos que habitualmente se dan por triviales o evidentes.

En lo que respecta a la formalización de la isomorfía entre cuerpos reales, puede verse el código completo tanto en el anexo como en Github [15].

Se contemplan las siguientes opciones como posibles mejoras a futuro o como continuación del trabajo realizado en Lean4:

- Optimizar la cantidad de líneas de código utilizadas gracias a la experiencia obtenida escribiendo en Lean4.
- Llevar un registro paralelo de resultados auxiliares demostrados con el objetivo de evitar redundancias.
- Consolidar el código para conseguir que pueda ser reutilizado en versiones posteriores de Lean. Por ejemplo, tácticas como `simp` tienen asociados resultados marcados en Mathlib con `[@simp]`. En futuras versiones de Mathlib, estos resultados podrían cambiar, dejando algunas pruebas realizadas en la versión actual incompletas.
- Búsqueda y demostración de resultados aún no registrados en Mathlib.

Bibliografía

- [1] J. Avigad, L. De Moura, S. Kong, S. Ullrich, and contributors from the Lean Community. Theorem proving in lean 4, 2024. Ver https://leanprover.github.io/theorem_proving_in_lean4. Última consulta: Enero de 2026.
- [2] K. Buzzard. Formalising mathematics 2024. <https://github.com/ImperialCollegeLondon/formalising-mathematics-2024>, 2024.
- [3] K. Buzzard and M. Pedramfar. The Natural Number Game. https://www.ma.imperial.ac.uk/~buzzard/xena/natural_number_game/. Consultado por última vez en Septiembre de 2025.
- [4] J. Chávez and J. Huamán. Teoría de tipos dependientes: Una primera aproximación. *Pesquimat*, 26(1):71–87, 2023.
- [5] A. Church. A formulation of the simple theory of types. *The journal of symbolic logic*, 5(2):56–68, 1940.
- [6] L. P. Community. Lean prover community. Ver <https://leanprover-community.github.io/>. Última consulta: 19 de enero de 2025.
- [7] L. P. Community. mathlib4: The lean4 mathematical library. Ver <https://github.com/leanprover-community/mathlib4>. Última consulta: 3 de diciembre de 2025.
- [8] H. B. Curry, R. Feys, W. Craig, J. R. Hindley, and J. P. Seldin. *Combinatory logic*, volume 1. North-Holland Amsterdam, 1958.
- [9] G. Gao, H. Ju, J. Jiang, Z. Qin, and B. Dong. A semantic search engine for mathlib4. *arXiv preprint arXiv:2403.13310*, 2024. Ver <https://leansearch.net/>. Última consulta: Noviembre de 2025.
- [10] K. Gödel. Über formal unentscheidbare sätze der principia mathematica und verwandter systeme i. *Monatshefte für mathematik und physik*, 38(1):173–198, 1931.
- [11] D. Hilbert. *Die Grundlagen der Mathematik*, pages 1–21. Vieweg+Teubner Verlag, Wiesbaden, 1928.
- [12] W. A. Howard et al. The formulae-as-types notion of construction. *To HB Curry: essays on combinatory logic, lambda calculus and formalism*, 44:479–490, 1980.
- [13] S. C. Kleene and J. B. Rosser. The inconsistency of certain formal logics. *Annals of Mathematics*, 36(3):630–636, 1935.
- [14] L. Kovács and A. Voronkov. The vampire theorem prover, 2024. Ver : <https://vprover.github.io/>.
- [15] M. Laborda Velázquez. Proyecto, 2025-2026. Ver : <https://github.com/Miiike1/proyecto>.
- [16] P. Montero. Formalización de las matemáticas con lean. un caso de estudio: Resultados de topología general, 2025. TFG en Matemáticas, UCM. Ver en: <https://github.com/pepamontero/tfg-topologia-lean4/blob/main/Memoria/main.pdf>.

- [17] Morph. MoogLe, a semantic search engine for mathlib, the lean mathematical library, 2023. Ver <https://www.moogLe.ai/>. Última consulta: Noviembre de 2025.
- [18] T. Nipkow, L. C. Paulson, and M. Wenzel. *Isabelle/HOL — A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002.
- [19] J. G. Perdomo, W. C. Rojas, and A. C. López. Programacion funcional y lambda cálculo. *Ingeniería e Investigación*, (40):72–82, 1998.
- [20] S. Schulz. The e automated theorem prover. Ver: <https://www.lehre.dhbw-stuttgart.de/~sschulz/E/E.html>. Última consulta: 3 de diciembre de 2025.
- [21] The Coq Development Team. *The Coq Proof Assistant Reference Manual*, 2024. Ver: <https://rocq-prover.org/>.
- [22] A. M. Turing. Computing machinery and intelligence (1950). *Mind*, 59(236):33–60, 2021.